

23. Bölüm

Varlık Çerçevesi

23.1 Nesne-İlişkili Eşleme

Kurumsal yazılımların işlendiği 19 başlığında katmanlı mimarinin bakımı kolaylaştırdığı ve ölçeklenebilirliği artırdığı anlatılmıştı. Ayrıca aynı başlık altında katmanların nasıl tanımlanması gerektiği 19.4.5 başlığında detaylı anlatılmıştı. İş katmanındaki veri taşıyan sınıfların (BEC) kullanılarak veri erişim katmanının tasarlandığı ve iş varlıklarının soyut olarak veri tabanında nasıl saklanacağına veri erişim bileşenleri (DAC) ile tanımlandığı anlatılmıştı. Tüm bunlara örnek olarak 19.8 başlığında bir uygulama örneği verilmişti.

Veri erişim katmanında yer alacak her veri erişim bileşenlerinin (DAC) her biri, yaratma (**create**), veri tabanında kayıt (**record**), okuma (**read**), güncelleme (**update**) ve silme (**delete**) işlemini gerçekleştirilecek şekilde yöntemlere sahiptir. Günümüzde iş varlığı bileşenlerini (BEC) için bu CRUD yöntemlerini otomatik sağlayan ve en az kodlama ile eşleştiren teknolojiler ortaya çıkmıştır. Bu teknolojinin adı **nesne-ilişkili eşlemedir** ORM (**Object-Relational Mapping**). Kısaca iş varlıklarımızı veri tabanındaki tablolardaki kayıtlarla eşler. Yani projemizde CRUD işlevleri için gerekli SQL komutları ORM tarafından otomatik olarak oluşturulur ve bu teknoloji sayesinde daha hızlı ve hatasız yazılım geliştirilir. ORM, çok fazla programlama yapmadan, otomatik bir şekilde iş nesnelerinden ilişkisel veri tabanına veri depolamak için bir araçtır. Üç ana bölümden oluşur:

1. İş bileşenlerini içeren etki alanı sınıf nesneleri
2. İlişkisel veri tabanı nesneleri
3. Etki alanı nesnelerinin ilişkisel veri tabanı nesnelere nasıl eşlendiğine dair eşleme bilgileri.

ORM, veri tabanı tasarımımızı iş bileşeni sınıf tasarımımızdan ayrı tutmamızı sağlar. Bu, uygulamayı sürdürülebilir ve genişletilebilir hale getirir. Ayrıca standart CRUD işlemini (Oluşturma, Okuma, Güncelleme ve Silme) otomatikleştirir, böylece geliştiricinin bunu manuel olarak yazmasına gerek kalmaz. Veri erişimi için, veri erişim katmanında detaylı ADO.NET kodunu yazmak ve yönetmek sıkıcı ve monoton bir iştir. Microsoft, uygulamanız için veri tabanı ile ilgili faaliyetleri otomatikleştirmek amacıyla **varlık çerçevesini** (**Entity Framework**-kısaca **EF**) geliştirmiştir.

Örnek Kurumsal Uygulama 19.8 başlığında verilen örnekte ilk önce veri tabanında tablo oluşturulmuş, daha sonrasında bu tabloya veri ekleme ve veri çekme işlemi kodlanmıştı. Bu yaklaşıma **önce veri tabanı** (**database first**) adı verilir. Önce veri tabanı yaklaşımında, veri tabanı ile iş varlıklar (BEC) eşleşmesi için ilk önce veri tabanı tabloları tanımlanır ve sonrasında bu tablolar ile varlıklar eşleştirilir (**mapping**). Bu da yazılımın devreye alınacağı zaman, veri tabanı yöneticisine bir iş çıkarır. Yazılımın devreye alınmasında ve sonraki uygulama sürümlerinde kodlamayı yapan yazılımcının veri tabanı yöneticisi ile çalışmasını gerekir.

Önce veri tabanı yaklaşımında genel olarak **EDMX** (**Entity Data Model XML**) dosyaları kullanılır. Bu dosyalar temel olarak mevcut bir veri tabanı üzerinden görsel olarak .NET nesneleri arasındaki ilişkiyi tanımlayan bir XML belgesidir. Visual Studio içinde **.edmx** dosyası açıldığında genellikle bir grafik tasarım (**designer**) ekranı ile gösterilir. Burada tablolar, ilişkiler ve varlıklar (**entity**) sürükleyip bırakarak düzenlenebilir.

Scaffold-DbContext

EF Core, **.edmx** dosyasını desteklemez! Ancak **önce veri tabanı** yaklaşımı **EF Core** tarafında bu işlem **Scaffold-DbContext** komutuyla yapılır. Yeni kurumsal uygulama projelerinde genellikle **Entity Framework Core tercih edilir** ve veritabanında tablo oluşturmadan işe başlanır. Bu yaklaşıma **önce kod** (**code first**) yaklaşımı adı verilir.

Açık olan projemizde **EF Core** bileşenlerini **SQLite** veritabanı ile birlikte dahil etmek için **Tools → NuGet Package Manager → Package Manager Console** üzerinden komut vererek bileşenleri projemize dahil ederiz;

```
Install-Package Microsoft.EntityFrameworkCore
Install-Package Microsoft.EntityFrameworkCore.Sqlite
```

23.2 Veri Tabanı Bağlamı

Veri Tabanı Bağlamı (**DbContext**), kullanmayı öğrenmemiz gereken ilk ve en önemli **EF veri tipidir**. Veri tabanı sorguları için bir ara yüz görevi görür ve nesnelerde yaptığınız değişiklikleri takip eder, böylece veriler

veri tabanında kalıcı hale getirilebilir.

Nesnelerini kullanarak verileri sorgulamak, eklemek, güncellemek ve silmek için EF kullanmak amacıyla öncelikle modelinizde tanımlanan varlıkları ve ilişkileri bir veri tabanındaki tablolara eşleyen bir model (BEC) oluşturmanız gerekir. Aşağıdaki **Öğrenci** ve **Bölüm** varlığını ele alalım;

```
public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public int Yaş { get; set; }
}
public class Bölüm
{
    public int BölümNo { get; set; }
    public string Ad { get; set; }
}
```

Bir modeliniz olduğunda, iş varlığınızın uygulamanızın etkileşime girdiği ilk sınıf **bağlam sınıfıdır** (**context class**). **Microsoft.EntityFrameworkCore.DbContext** bağlam sınıfını şu amaçlarla kullanabilirsiniz:

- ◊ Sorguları yazıp çalıştırabilirsiniz.
- ◊ Sorgu sonuçlarını varlık nesneleri olarak somutlaştırabilirsiniz
- ◊ Varlık nesnelerinde yapılan değişiklikleri izleyebilirsiniz.
- ◊ Nesne değişikliklerini veri tabanında kalıcı hale getirebilirsiniz.
- ◊ Nesneleri bellekte kullanıcı arayüzü (UI) bileşenlerine bağlayabilirsiniz.

Bağlam sınıfıyla çalışmanın önerilen yolu, **DbContext** sınıfından türetilen bağlam sınıfında, belirtilen varlıkların koleksiyonlarını temsil eden **DbSet** özelliklerini açığa çıkaran bir sınıf tanımlamaktır. **EF Core**, önce kod (**code first**) yaklaşımını kullanır. Bu nedenle bağlam sınıfını programcı yazar;

```
public class OkulContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }
    public DbSet<Bölüm> Bölümler { get; set; }
}
```

Bir bağlam sınıfınız olduğunda, bu özellikler **DbSet** veri tipine sahip olduğundan bağlamdaki varlıkları sorgulayabilir, ekleyebilir veya silebilirsiniz. Bir bağlam nesnesindeki **DbSet** özelliğine eriştiğinizde, belirtilen tipteki tüm varlıkları döndüren bir başlangıç sorgusunu çalışır. Bu sorgu sonucu;

- ◊ Numaralandırılan nesne listesi **foreach** ile gezilebilir.
- ◊ Numaralandırılan nesne listesi **ToArray()**, **ToDictionary()** ve **ToList()** gibi yöntemlerle koleksiyonlara çevrilebilir.
- ◊ **First** veya **Any** gibi LINQ genişleme yöntemleri (**LINQ extension method**) ile sorgulanabilir.
- ◊ Bir varlık, bağlamda yüklü değilse **DbEntityEntry.Reload** ve **Database.ExecuteSqlCommand** ile yeniden yüklenebilir.

Bağlam nesnesinin ömrü, örneğin (**instance**) oluşturulmasıyla başlar ve örneğin atılması veya çöp toplanmasıyla sona erer. Bunun için **using** anahtar kelimesi kullanılır;

```
using (var context = new AppDbContext())
{
    var öğrenciler = context.Öğrenciler;
    foreach (var o in öğrenciler)
    {
        Console.WriteLine($"Ad: {o.Ad}, Yaş: {o.Yaş}");
    }
}
```

Burada veritabanından veriler çekilirken üç yöntem kullanılır;

§23.2.1 Önden Yükleme

Önden yükleme (**eager loading**), ana veriyle birlikte ilişkili verilerin de tek bir SQL sorgusunda (**JOIN** kullanarak) veritabanından çekilmesidir. **EF Core**'da bu işlem **.Include()** metodu ile yapılır. İlişkili verilere mutlaka ihtiyaç duyacağınızdan eminseniz ve veritabanına gidiş-dönüş (**round-trip**) sayısını azaltmak istiyorsanız tercih edilir.

```
using (var context = new KitaplikContext())
```

```
{
    // Yazarı çekerken kitaplarını da aynı sorguda getirir
    var yazar = context.Authors
    .Include(a => a.Books)
    .FirstOrDefault(a => a.AuthorId == 1);
    foreach (var kitap in yazar.Books)
    {
        Console.WriteLine(kitap.Title);
    }
}
```

§23.2.2 Tembel Yükleme

Tembel yükleme (**lazy loading**), ilişkili verilerin sadece onlara erişildiği anda (kod içerisinde özellik çağrıldığında) veritabanından çekilmesidir.

EF Core'da bu özelliği kullanmak için; `Microsoft.EntityFrameworkCore.Proxies` paketini kurmalı ve navigasyon özelliklerini **virtual** yapmalısınız. İlişkili verilere her zaman ihtiyaç duyulmadığı durumlarda tercih edilir. Ancak döngü içerisinde kullanılırsa, her yazar için ayrı bir SQL sorgusu atılması gerektiğinden "N+1 Sorgu Problemine" yol açabilir.

```
// Context yapılandırmasında: options.UseLazyLoadingProxies(); yazmalıyız.
using (var context = new KitaplikContext())
{
    // Sadece yazar bilgisi çekilir
    var yazar = context.Authors.FirstOrDefault(a => a.AuthorId == 1);
    // Aşağıdaki satır çalıştığı anda EF Core arka planda yeni bir SQL sorgusu atarak kitapları
    // → getirir
    foreach (var kitap in yazar.Books)
    {
        Console.WriteLine(kitap.Title);
    }
}
```

§23.2.3 Açıkça Yükleme

Açıkça Yükleme (**explicit loading**), ilişkili verilerin ana veri çekildikten sonra, ihtiyaç duyulduğu anda manuel olarak (açık bir komutla) yüklenmesidir. Lazy Loading'e benzer ancak verinin ne zaman çekileceği tamamen programcının kontrolündedir. Nesne çekildikten sonra belirli bir mantığa göre ilişkili veriyi çekip çekmemeye karar verecekseniz idealdir.

```
using (var context = new KitaplikContext())
{
    var yazar = context.Authors.FirstOrDefault(a => a.AuthorId == 1);
    // Belirli bir koşula göre kitapları daha sonra yüklüyoruz
    if (yazar.IsActive)
    {
        context.Entry(yazar)
            .Collection(a => a.Books)
            .Load(); // Veritabanına şimdi gidilir
    }
}
```

§23.2.4 Varlıkları Yüklerken Değişiklik Takibini Kapatma

EF Core'da `AsNoTracking()` yöntemi, sorgu sonucunda dönen nesnelerin **değişiklik için takip edilmesini** kısıtlar. Özellikle sadece veri okuma (**read-only**) işlemlerinde bellek (RAM) ve işlemci (CPU) yükünü azaltmak için kullanılır.

```
using (var context = new MyDbContext())
{
    // Verileri takip mekanizmasına (Change Tracker) almadan çeker
    var musteriler = context.Customers
        .AsNoTracking()
        .ToList();
    // Performans avantajı: Bellekte nesnelerin bir kopyası (snapshot) tutulmaz.
}
```

23.3 Önce Kod

Önce kod (**code first**) yaklaşımında ise geliştirici, modeli (BEC) belirtmek için kod yazar. EF, geliştirici tarafından kodlanan varlık sınıflarına ve ek model yapılandırmasına bağlı olarak ver modellerini ve veri tabanı eşlemelerini **çalışma zamanında** (**run-time**) üretir. Kısaca EF, **kodlanan varlık sınıflarından veri tabanını otomatik olarak oluşturur**. Yani eşleme (**mapping**) otomatik olarak yapılır.

Bu yaklaşım, veri tabanının tasarlandığı **.edmx** dosyası kullanmadan veri tabanı tablosundaki kayıtlar ile varlıklarımız otomatik olarak eşleme yapılır. Ayrıca mevcut veri tabanı tablolarını da kullanabilirsiniz, buna **mevcut veri tabanı ile önce kod** (**code first with existing database**) denir. Aşağıdaki gibi bir BEC varlığımız olsun;

```
public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}
```

Şimdi varlık sınıfımız ve veri tabanınız arasındaki köprü olan bağlamımız olan **DbContext** sınıfı tasarlayacağız. Bu sınıfa da varlık sınıfına ait bir **DbSet<>** özelliği eklenir;

```
public class AppDbContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }
}
```

Artık veri tabanına ilk bağlandığımız anda **Öğrenci** tablosu veri tabanında oluşacaktır; Bunu yapmak için aşağıdaki program yazılabilir,

```
using (var context = new AppDbContext())
{
    var yeniÖğrenciler = new[]
    {
        new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
        new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
        new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
    };
    context.Öğrenciler.Add(yeniÖğrenciler[0]);
    context.Öğrenciler.Add(yeniÖğrenciler[1]);
    context.Öğrenciler.Add(yeniÖğrenciler[2]);
    // ya da context.Öğrenciler.AddRange(yeniÖğrenciler);
    context.SaveChanges();
}
```

Bu program bu haliyle çalışmaz! Çünkü veri tabanı bağlantı kısmı eksiktir. Ama genel eşleşme iskeleti bu şekildedir. **SQLite** veri tabanı kullanarak buna çalışan bir örnek verelim. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
EFCoreUygulama/ → Çözüm (Solution)
├─ CodeFirst1/ → Proje (Console Application)
│   └─ Program.cs → Ana (Main) Program
```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File→New→Project...** seçilerek **EFCoreUygulama** adlı çözüm içinde **CodeFirst1** adlı ilk projemizi **Console Application** olarak oluşturulur.
2. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır; **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.Sqlite** paketleri yüklenir.
3. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```
using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.Sqlite; // Install Nuget package

using (var context = new AppDbContext())
{
```

```

context.Database.EnsureCreated();
Console.WriteLine("SQLite Veri Tabanı Üzerinde Tablolar Oluşturuldu!");
var yeniÖğrenciler = new[]
{
    new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
    new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
    new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
};
context.Öğrenciler.AddRange(yeniÖğrenciler);
context.SaveChanges();

var öğrenciler = context.Öğrenciler;
Console.WriteLine("Veritabanındaki Öğrenciler:");
foreach (var o in öğrenciler)
{
    Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
}

var öğrenci = context.Öğrenciler.FirstOrDefault(u => u.Ad == "İlhan");
if (öğrenci != null)
{
    context.Öğrenciler.Remove(öğrenci);
    context.SaveChanges();
    Console.WriteLine($"'{öğrenci.Ad}' adlı öğrenci silindi.");
}
context.Database.CloseConnection();
//context.Database.EnsureDeleted();
}

public class AppDbContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        var çalışmaKlasörü=AppDomain.CurrentDomain.BaseDirectory;
        //Üç klasör geri git. Orada CodeFirst.sqlite veritabanını oluştur.
        var dbKlasörü =Path.Combine(çalışmaKlasörü, "..", "..", "..","CodeFirst.sqlite");
        //Bunu yapınca Solution Explorer üzerinde veritabanı görünür.
        optionsBuilder.UseSqlite($"Data Source={dbKlasörü}");
    }
}

public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}

```

4. Program çalışıp `context.Database.EnsureCreated()`; icra edildiği anda bağlantı kurulan `SQLite` veri tabanında `Öğrenciler` tablosu oluşur (Şekil 23.1a).

5. Aynı projede `CodeFirst.sqlite` veritabanını silindikten sonra bir başka sınıf olan `Kullanıcı` sınıfı eklenebilir.

```

using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.Sqlite; // Install Nuget package

var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
//Üç klasör geri git. Orada CodeFirst.sqlite veritabanını oluştur.
var dbKlasörü = Path.Combine(çalışmaKlasörü, "..", "..", "..", "CodeFirst.sqlite");
//Bunu yapınca Solution Explorer üzerinde veritabanı görünür.
var options = new DbContextOptionsBuilder<AppDbContext>()

```

```

.UseSqlite($"Data Source={dbKlasörü}")
.Options;

using (var context = new AppDbContext(options))
{
    context.Database.EnsureCreated();
    Console.WriteLine("SQLite Veri Tabanı Üzerinde Tablolar Oluşturuldu!");
    var yeniÖğrenciler = new[]
    {
        new Öğrenci { Ad = "İlhan", Soyad="ÖZKAN", Yaş = 30 },
        new Öğrenci { Ad = "Mehmet", Soyad="YILMAZ", Yaş = 25 },
        new Öğrenci { Ad = "Abdullah", Soyad="YILDIZ", Yaş = 28 },
    };
    context.Öğrenciler.AddRange(yeniÖğrenciler);
    context.SaveChanges();

    var öğrenciler = context.Öğrenciler;
    Console.WriteLine("Veritabanındaki Öğrenciler:");
    foreach (var o in öğrenciler)
    {
        Console.WriteLine($"Adı: {o.Ad}, Soyadı: {o.Soyad}, Yaşı: {o.Yaş}");
    }

    var öğrenci = context.Öğrenciler.FirstOrDefault(u => u.Ad == "İlhan");
    if (öğrenci != null)
    {
        context.Öğrenciler.Remove(öğrenci);
        context.SaveChanges();
        Console.WriteLine($"'{öğrenci.Ad}' adlı öğrenci silindi.");
    }
    var kullanıcılar = context.Kullanıcılar;
    Console.WriteLine("Veritabanındaki Kullanıcılar:");
    foreach (var o in kullanıcılar)
    {
        Console.WriteLine($"Adı: {o.Ad}, Id: {o.Id}");
    }

    context.Database.CloseConnection();
    //context.Database.EnsureDeleted(); Veritabanı Siler
}

public class AppDbContext : DbContext
{
    public DbSet<Öğrenci> Öğrenciler { get; set; }
    public DbSet<Kullanıcı> Kullanıcılar { get; set; }

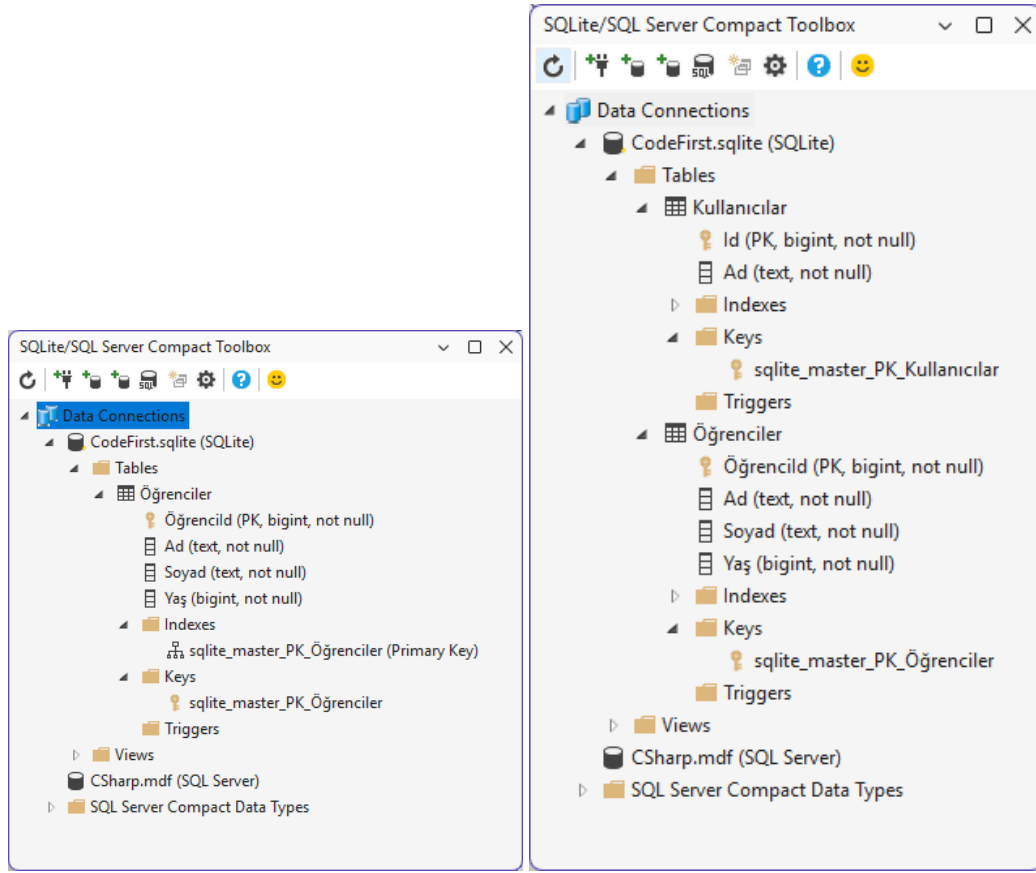
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
}

public class Öğrenci
{
    public int ÖğrenciId { get; set; }
    public string Ad { get; set; }
    public string Soyad { get; set; }
    public int Yaş { get; set; }
}

public class Kullanıcı
{
    public int Id { get; set; }
    public string Ad { get; set; }
}

```

6. Program çalışıp `context.Database.EnsureCreated()`; icra edildiği anda bağlantı kurulan SQLite veri



(a) Öğrenci Varlığı Eklenince

(b) Öğrenci ve Kullanıcı Varlığı Eklenince

Şekil 23.1: CodeFirst.sqlite Veritabanında EF Core Tarafından Oluşturulan Tablolar

tabanında **Öğrenciler** yanında **Kullanıcılar** tablosu oluşur (Şekil 23.1b).

7. Bu sefer veritabanına bağlanma `OnConfiguring()` yöntemi ile değil başka şekilde başlatılmıştır. Yeni programda `AppDbContext` nesnesine `DbContextOptionsBuilder` parametresi verilerek başlatılmıştır.

ID Karakterleri İçeren Özellikler

`CodeFirst.sqlite` veritabanında oluşan tabloların isimleri `AppDbContext` sınıfında tanımlanan `DbSet<>` özelliklerinden gelmiştir. Ayrıca veri tabanı incelendiğinde `ÖğrenciId` ve `Id` kolonlarının **birincil anahtar** (**primary key**) olduğu da görülmektedir. Bu alanlar `User` sınıfındaki `Id` ve `Öğrenci` sınıfındaki `ÖğrenciId` özellik kimliklerinin sonunda `Id` karakterleri içeriyor olmasından kaynaklanmaktadır. Varlık sınıfları tasarlanırken göz önüne alınan bunun gibi birçok **kimliklendirme kuralı** (**convention**) vardır.

23.4 Önce Kod Kimliklendirme Kuralları

Varlık sınıfında özellik ve alan kimliklendirmesi oluşturulacak veri tabanındaki tabloları etkiler. Dolayısıyla yukarıda anlatılan **kimliklendirme kuralları** (**convention**) EF Core'un kalbini oluşturur.

§23.4.1 Var Olan Bir Kimliklendirme Kuralını Geçersiz Kılma

`ConfigureConventions` ile tanımlanmış herhangi bir kuralı (**convention**), `OnModelCreating()` yöntemini geçersiz kılarak kaldırabilirsiniz.

Aşağıda veri tabanında, varlığa karşılık oluşturulan, tablo isimleri sonuna "s"/"ler" eki getirilme kaldırılmıştır.

```
public class DersContext : DbContext
{
    public DbSet<Saat> Saatler { set; get; }
    protected override void OnModelCreating(DbModelBuilder modelBuilder)
    {
        modelBuilder.Conventions.Remove<PluralizingTableNameConvention>();
    }
}
```

```

    // PluralizingTableNameConvention adlı kural kaldırılmıştır.
}
}

```

§23.4.2 Birincil Anahtar Kuralı

Son örneğimizde olduğu gibi varsayılan olarak, bir sınıftaki bir özelliğin;

- ◊ Kimliği "ID" ise
- ◊ Kimlik "ID" karakterleri ile bitmişse
- ◊ Sınıf adını "ID" ile takip edilmişse

bu özellik **birincil anahtardır** (**primary key**).

Kimlikteki "ID" karakterleri büyük/küçük harfe duyarlı değildir. "iD", "Id" olabilir. Ayrıca birincil anahtar olacak özelliğinin veri tipi sayısal (**int**, **long**) veya **GUID** olabilir;

```

public class Okul
{
    public int okulId { get; set; } // Primary key
    //...
}

```

§23.4.3 Veri Tipi Keşfi

EF Core (**Entity Framework Core**) içerisinde **veri tipi keşfi** (**type discovery**), modelinizin veritabanı şemasına nasıl dönüştürüleceğini belirleyen kritik bir mekanizmadır. Bir sınıfın veritabanında bir tabloya karşılık gelip gelmeyeceğine EF Core belirli kurallar çerçevesinde karar verir.

İşte EF Core'un veri tiplerini keşfetme ve modele dahil etme hiyerarşisi:

DbSet<T> Özellikleri

En temel keşif yöntemi, **DbContext** sınıfınız içinde tanımladığınız **DbSet<TEntity>** özellikleridir. EF Core, bu özellikleri tarar ve belirtilen tüm veri tiplerini modelin bir parçası olarak kabul eder.

```

public class UygulamaDbContext : DbContext
{
    // 'Kitap' veri tipi ve tüm bağlı olduğu alt veri tipleri keşfedilir.
    public DbSet<Kitap> Kitaplar { get; set; }
}

```

Gezinme Özellikleri

EF Core sadece **DbSet** içindekileri değil, bu sınıfların işaret ettiği diğer sınıfları da özyinelemeli (**recursive**) olarak keşfeder. EF Core'da buna **gezinme özelliği** (**navigation property**) adı verilir. Gezinme özelliği özyinelemeli(**recursive discovery**) olarak yapılır.

Aşağıdaki örnekten gidecek olursak, eğer **Kitap** sınıfınız içinde bir **Yazar** referansı varsa, **Yazar** için bir **DbSet** tanımlamasanız bile EF Core onu keşfeder ve bir tablo oluşturur.

```

public class Kitap
{
    public int Id { get; set; }
    public string Ad { get; set; }
    // EF Core bu navigasyon özelliğini takip ederek 'Yazar' türünü keşfeder.
    public Yazar Yazar { get; set; }
}

```

Buna ilişkin bir gerçek bir örnek **Microsoft SQL Server LocalDB** üzerinden verilecektir. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```

EFCoreUygulama/ → Çözüm (Solution)
├── CodeFirst2/ → Proje (Console Application)
│   ├── SQLServer.mdf → MSSQL Veritabanı (Data)
│   ├── SQLServer_log.ldf → MSSQL Veritabanı (Log)
│   └── Program.cs → Ana (Main) Program

```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File**→**New**→**Project...** seçilerek **EFCoreUygulama** adlı çözüm içinde **CodeFirst2** adlı ilk projemizi **Console Application** olarak oluşturulur.
2. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır ve


```

Microsoft.EntityFrameworkCore ve
Microsoft.EntityFrameworkCore.Sqlserver paketleri yüklenir.
3. Visual Studio üzerinde Tools→Connect To Database tıklanarak projenin klasöründe SQLServer.mdf
adında bir Microsoft SQL Server LocalDB veritabanı oluşturulur.
4. Proje ile oluşan Program.cs dosyasını aşağıdaki şekilde değiştirilir;
using Microsoft.EntityFrameworkCore; // Install Nuget package
using Microsoft.EntityFrameworkCore.SqlServer; // Install Nuget package

var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü=Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
var db = Path.Combine(projeKlasörü, "SQLServer.mdf");

string connectionString = $"Server=(LocalDB)\MSSQLLocalDB; AttachDbFilename={db};
→ Integrated Security=True;Connect Timeout=30;";
var options = new DbContextOptionsBuilder<AppDbContext>()
.UseSqlServer(connectionString)
.Options;

using (var context = new AppDbContext(options))
{
    context.Database.EnsureCreated();
    Console.WriteLine("SQServer Veri Tabanı Üzerinde Tablolar Oluşturuldu!");
    var yerleşkeler = context.Yerleşkeler;
    Console.WriteLine("Yerleşkeler:");
    foreach (var y in yerleşkeler)
    {
        Console.WriteLine($"Id: {y.Id}, Yerleşke Adı: {y.YerleşkeAdı}:");
        foreach (var b in y.Birimler)
        {
            Console.WriteLine($"Id: {b.Id}, Birim Adı: {b.BirimAdı}:");
        }
    }
    context.Database.CloseConnection();
}

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
    public DbSet<Yerleşke> Yerleşkeler { set; get; }
}

public class Yerleşke
{
    public int Id { set; get; }
    public string YerleşkeAdı { set; get; }
    public virtual ICollection<Birim> Birimler { set; get; }
}

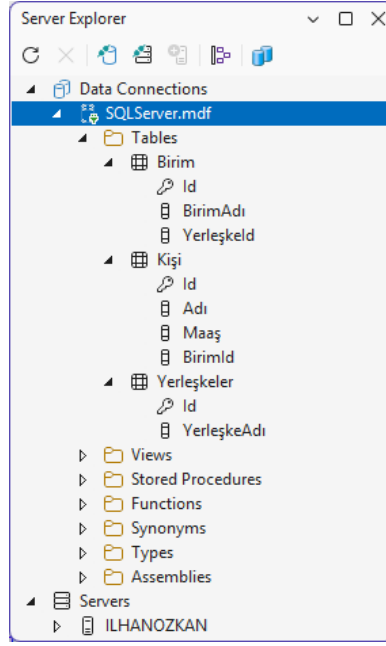
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> Personel { set; get; }
}

public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }
}

public class Müdür : Kişi
{
    public string MüdürÖzelliği { set; get; }
}

public class Şef : Kişi

```



Şekil 23.2: EF Core Veri Tipi Keşfi

```
{
    public string ŞefÖzelliği { set; get; }
}
public class Muhasebeci : Kişi
{
    public string MuhasebeciÖzelliği { set; get; }
}
```

5. Program çalıştığında oluşacak veri tabanı içeriği Şekil 23.2’de verilmiştir. Şekilde görüldüğü üzere `AppDbContext` sınıfında yalnızca `DbSet<Yerleşke> Yerleşkeler set; get;` özelliği tanımlanmasına rağmen veritabanında tüm varlıklara ilişkin tablolar oluşturulmuştur.

OnModelCreating İçinde Manuel Veri Tipi Dahil Etme

Bazen bir sınıfın ne `DbSet` olarak tanımlanmasını ne de bir navigasyon özelliğiyle erişilmesini istersiniz. Bu durumda `ModelBuilder` kullanarak türü manuel olarak modele zorla dahil edebilirsiniz.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Veri tipi manuel olarak keşif sürecine dahil edilir
    modelBuilder.Entity<LogKaydı>();
}
```

Keşfedilen Veri Tiplerinin Yapılandırılması

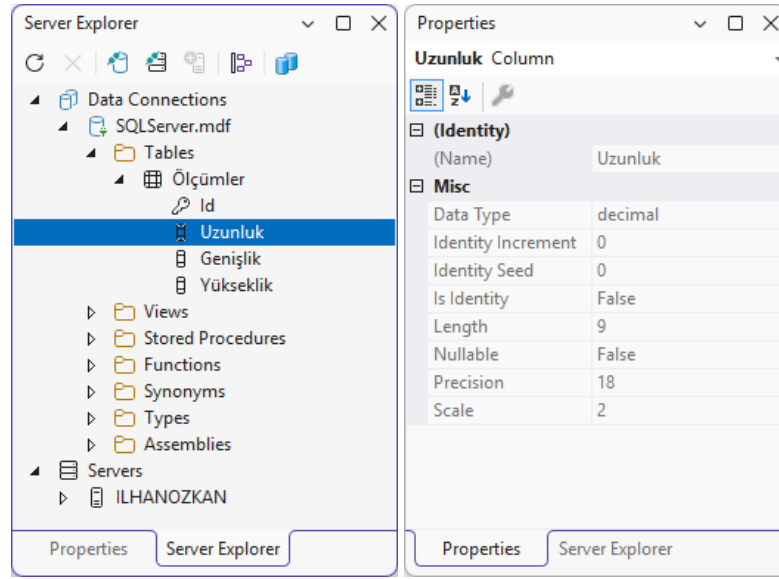
EF Core bir veri tipini keşfettiğinde, onun nasıl bir tabloya dönüşeceğini şu üç yöntemle belirler (Öncelik sırasına göre):

1. **Fluent API:** `OnModelCreating` içindeki tanımlamalar en yüksek önceliğe sahiptir.
2. **Veri Öznitelikleri (data annotations):** Sınıf üzerindeki `[Table]`, `[Key]`, `[Required]` gibi öznitelikler (`attribute`).
3. **Kimliklendirme Kuralları (conventions):** Varsayılan kurallar (Örn: `Id` kimlikli özelliklerin birincil anahtar yapılması gibi).

Veri Tipi Keşfini Engelleme (NotMapped)

Keşfedilmesini istemediğiniz sınıfları veya özellikleri dışarıda bırakmak için şu yöntemleri kullanabilirsiniz:

1. **Öznitelik ile:** Sınıfın üzerine `[NotMapped]` eklemek.
2. **Fluent API ile:** `modelBuilder.Ignore<GeciciTur>();` kullanmak.



(a) Ölçüm Tablosu

(b) Ölçüm Tablosu Uzunluk Özelliği

Şekil 23.3: Veritabanında Ölçüm için EF Core Tarafından Oluşturulan Tablo

Özet

EF Core, keşfettiği veri tiplerinde, sınıfta fiziksel olarak bulunmayan ancak veritabanında olması gereken sütunları da keşfedebilir. Örneğin, bir **yabancı anahtar (foreign key)** özelliğini sınıfa yazmazsanız, **EF Core** bunu bir gölge özellik (**shadow property**) olarak arka planda keşfeder ve yönetir.

Yöntem	Keşif Biçimi	Öncelik
<code>DbSet<T></code>	Doğrudan tanımlama.	Orta
<code>Navigasyon</code>	Özyinelemeli (Bağımlı veri tipleri).	Orta
<code>Fluent API</code>	Manuel (<code>modelBuilder.Entity()</code>).	En Yüksek
<code>Annotations</code>	Öznitelikler (<code>[Table]</code>).	Yüksek
<code>Conventions</code>	Varsayılan kimliklendirme kuralları.	Düşük

Tablo 23.1: EF Core Veri Tipi Keşfi ve Yapılandırma Hiyerarşisi

§23.4.4 Ondalık Kimliklendirme Kuralı

Ondalık kimliklendirme kuralı (decimal convention), ondalık sayıların (özellikle para birimleri ve hassas ölçümler) veritabanında hangi hassasiyetle (**precision**) ve ölçekle (**scale**) saklanacağını belirleyen kurallardır. Varsayılan olarak **EF Core**, **decimal** veri tipindeki bir alanı veritabanına aktarırken genellikle **decimal(18,2)** formatını kullanır. Bu, toplamda 18 basamak ve virgülden sonra 2 basamak anlamına gelir.

```
public class Ölçüm
{
    public int Id { set; get; }
    public decimal Uzunluk { set; get; }
    public decimal Genişlik { set; get; }
    public decimal Yükseklik { set; get; }
}
```

Bu varlığın veritabanında oluşmuş hali Şekil 23.3'de verilmiştir.

Finansal uygulamalarda bu varsayılan değer genellikle yetersiz kalır. Üç şekilde yapılır.

Genel Kural

EF Core 6.0+ sonrasında projenizdeki tüm **decimal** alanların aynı formatta (örneğin (18,4)) olmasını istiyorsanız, her varlık (entity) için tek tek ayar yapmak yerine genel bir kural tanımlayabilirsiniz.

```
protected override void ConfigureConventions(ModelConfigurationBuilder configurationBuilder)
{
    // Projedeki tüm decimal özellikler varsayılan olarak (18,4) olacak
    configurationBuilder.Properties<decimal>()
        .HavePrecision(18, 4);
}
```

```
}
```

Fluent API ile Yapılandırma

Sadece belirli bir sınıfın belirlenen bir özelliği için kural koymak istediğinizde kullanılır. Bu yöntem global kuralları ezer.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Kitap>()
        .Property(k => k.Fiyat)
        .HasPrecision(18, 2); // Toplam 18 basamak, 2 basamak virgülden sonra
}
```

Öznitelikler Kullanarak

Sınıf içindeki özelliklerin üzerine doğrudan öznitelik eklenerek (**data annotations**) yapılır.

```
public class Kitap
{
    public int Id { get; set; }
    [Precision(14, 3)]
    public decimal Agirlik { get; set; }
}
```

§23.4.5 İlişki Kuralı

İki varlık arasındaki ilişki, **gezinme özelliğini** (**navigation property**) kullanarak otomatik olarak yapılır. Bu gezinme özelliği basit bir referans tipi veya koleksiyon tipi olabilir.

```
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }

    //Telefon için gezinme(navigability) özelliği
    public ICollection<Telefon> Telefonlar { set; get; }
}

public class Telefon
{
    public int Id { set; get; }
    public string TelefonNumarası { set; get; }
    public string TelefonTürü { set; get; }
}
```

Buna ilişkin örnek 23.4.3 başlığında verilmiştir.

§23.4.6 Yabancı Anahtar Kimliklendirme Kuralı

Eğer A sınıfı B sınıfıyla ilişkiliyse ve B sınıfı, A'nın birincil anahtarıyla aynı ada ve tipe sahip bir özelliğe sahipse, EF Core otomatik olarak bu özelliğin **yabancı anahtar** (**foreign key**) olduğunu varsayar.

```
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> Personel { set; get; }
}

public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    public decimal Maaş { set; get; }
    public ICollection<Telefon> Telefonlar { set; get; }
    public int BirimId { set; get; } //Foreign Key
    public virtual Birim Birim { set; get; } //Foreign Key
}
```

```
public class Telefon
{
    public int Id { set; get; }
    public string TelefonNumarası { set; get; }
    public string TelefonTürü { set; get; }
}
```

Yukarıdaki sınıfların veri tabanındaki karşılığı aşağıdaki şekilde verilmiştir. **Server Explorer** üzerinde **Kişiler** tablosu sağ tıklanıp **Open Table Definition** seçildiğinde tablonun SQL kodu aşağıdaki gibi olduğu görülecektir;

```
CREATE TABLE [dbo].[Kişiler] (
    [Id] INT IDENTITY (1, 1) NOT NULL,
    [Adı] NVARCHAR (MAX) NOT NULL,
    [Maaş] DECIMAL (18, 2) NOT NULL,
    [BirimId] INT NOT NULL,
    CONSTRAINT [PK_Kişiler] PRIMARY KEY CLUSTERED ([Id] ASC),
    CONSTRAINT [FK_Kişiler_Birim_BirimId] FOREIGN KEY ([BirimId]) REFERENCES [dbo].[Birim] ([Id])
    → ON DELETE CASCADE
);
```

23.5 Önce Kod Veri Öznitelikleri

Veri öznitelikleri (**data annotations**), **EF Core**'ü veri tabanında tabloları ve kolonları oluştururken ve işlerken yönlendirir. Aşağıda çok kullanılan özniteliklere yer verilmiştir¹.

§23.5.1 [Column] Özniteliği

EF'ye, özelliğin adını kullanmak yerine belirli bir sütun adını kullanmasını söyler.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }
    [Column("Adı ve Soyadı")] //Kolon adı Adı yerine "Adı ve Soyadı" olsun
    public string Adı { set; get; }
}
```

§23.5.2 [Required] Özniteliği

Bir etki alanı sınıfının (BEC) bir özelliğine uygulandığında, veri tabanı sütunu **NOT NULL** olarak oluşturacaktır.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }

    [Required] //Adı Girilmeden Kişiye kayıt atılmaz!.
    public string Adı { set; get; }
}
```

§23.5.3 [MaxLength] ve [MinLength] Öznitelikleri

[MaxLength(int)] veya [MinLength(int)] özniteliği bir sınıfının (BEC) metin (**string**) alanına veya metin özelliğine uygulanabilir. **EF Core**, bir sütunun boyutunu belirtilen değere ayarlayacaktır.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Kişi
{
    public int Id { set; get; }
    [MinLength(3), MaxLength(100)] //Adı enz 3 krakter en fazla 100 karakter olabilir!.
    public string Adı { set; get; }
}
```

¹<https://learn.microsoft.com/en-us/ef/ef6/modeling/code-first/data-annotations>

§23.5.4 [ForeignKey(string)] Özniteliği

Aşağıdaki Örneği ele alalım;

```
using System.ComponentModel.DataAnnotations.Schema;
public class Müşteri
{
    public int Id { get; set; }
    [Required, MaxLength(100)]
    public string MüşteriAdı { get; set; }
    public int PostaAdresID { get; set; }
    [ForeignKey("PostaAdresID")]
    public virtual Adres PostaAdresi { get; set; }
}
public class Adres
{
    public int Id { get; set; }
    public string Sokak { get; set; }
    public string DışKapıNo { get; set; }
    public string İçKapıNo { get; set; }
    public string İlçe { get; set; }
    public string İl { get; set; }
    public long UlusalAdresKodu { get; set; }
}
```

ForeignKey nitelikleri olmadan, EF Core bunları karıştırabilir ve **PostaAdresi** alırken başka **ID** içeren özellikleri kullanabilir. Bunu önlemek için bu nitelik kullanılır.

§23.5.5 [InverseProperty(string)] Özniteliği

InverseProperty, iki varlık arasında birden fazla iki yönlü ilişki olduğunda iki yönlü ilişkileri tanımlamak için kullanılabilir. **EF Core**'e diğer taraftaki özelliklerle hangi gezinme (**navigation**) özelliklerini eşleştirmesi gerektiğini söyler. EF Core, iki varlık arasında birden fazla çift yönlü ilişki olduğunda diğer taraftaki hangi gezinme özelliğinin hangi özelliklerle eşleştiğini bilmez. İlgili sınıftaki karşılık gelen gezinme özelliğinin adını parametresi olarak alması gerekir.

```
using System.ComponentModel.DataAnnotations; //gerekli
public class Birim
{
    public int Id { set; get; }
    public string BirimAdı { set; get; }
    public virtual ICollection<Kişi> AsılPersonel { set; get; }
    public virtual ICollection<Kişi> GörevliPersonel { set; get; }
}
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
    [InverseProperty("AsılPersonel")]
    public virtual Birim AnaBirim { get; set; }
    [InverseProperty("GörevliPersonel")]
    public virtual Birim GörevliOlduğuBirim { get; set; }
}
```

Ağaç yapısında nesneler için bu nitelik kullanılabilir;

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;
public class AğaçDüğümü
{
    public int ID { get; set; }
    public int ParentID { get; set; }
    [ForeignKey("ParentID")]
    public AğaçDüğümü EbeveynDüğüm { get; set; }
}
```



```
[InverseProperty("ParentNode")]
public virtual ICollection<AğacDüğümü> ÇocukDüğümler { get; set; }
}
```

§23.5.6 [ComplexType] Özniteliği

Alan odaklı tasarımda DDD (Domain Driven Design) değer nesneleri (value object) gibi karmaşık tipler kendi başlarına izlenemezler ancak bir varlığın parçası olarak izlenirler. Böyle durumda bu nitelik kullanılır. Veri tabanını oluşturan kod içeriği aşağıda verilmiştir;

```
[ComplexType]
public class BlogDetay
{
    public DateTime? OluşturmaZamanı { get; set; }
    [MaxLength(250)]
    public string Açıklama { get; set; }
}
public class Blog
{
    public int Id { get; set; }
    public string Başlık { get; set; }
    public string İçerik { get; set; }
    public BlogDetay Detay { get; set; }
}
```

§23.5.7 [Range(min,max)] Özniteliği

İş varlığına ilişkin sayısal alanlarda sınır belirlemek için kullanılır;

```
public class MezuniyetDerecesi
{
    public int DereceID { get; set; }
    [Range(0, 4)]
    public Nullable<decimal> Derece { get; set; }
}

public class Öğrenci
{
    public int Id { get; set; }
    [Required, MaxLength(50)]
    public string Adı { get; set; }
    [Required, MaxLength(50)]
    public string Soyadı { get; set; }
    [EmailAddress, MaxLength(150)]
    public string EPosta { get; set; }
    public MezuniyetDerecesi MezuniyetDerecesi { get; set; }
}
```

§23.5.8 [NotMapped] Özniteliği

Önce kod yaklaşımında EF Core, desteklenen bir veri türünde olan her genel özellik için bir sütun oluşturur. [NotMapped] niteliği, bir veri tabanı tablosunda sütun istemediğimiz tüm özelliklere uygulanmalıdır.

```
public class Öğrenci
{
    public int Id { get; set; }
    [Required, MaxLength(50)]
    public string Adı { get; set; }
    [Required, MaxLength(50)]
    public string Soyadı { get; set; }

    public string TamAdı => string.Format("{0} {1}", Adı, Soyadı);

    [Required, MaxLength(150)]
    public string EPosta { get; set; }
    [NotMapped]
```

```
public float Ortalama { set; get; }
}
```

§23.5.9 [Table] Özniteliği

İş varlığına ilişkin veri tabanında tablo adı belirlemek için kullanılır.

```
[Table("Personel")]
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }
}
```

§23.5.10 [Index] Özniteliği

Veri tabanında oluşan kolonlar için dizin (**index**) oluşturmak için kullanılır;

```
public class Kişi
{
    public int Id { set; get; }
    public string Adı { set; get; }

    [Index]
    public int Yaş { set; get; }
}
```

İstenirse dizine ad da verilebilir;

```
[Index("IX_Kişi_Yaşı")]
public int Yaş { set; get; }
```

İstenirse tekil (**unique**) dizin de oluşturulabilir;

```
[Index(IsUnique = true)]
public int Yaş { set; get; }
```

Ayrıca birden fazla kolon kullanılarak da dizin oluşturulabilir;

```
[Index("IX_Kişi_AdveYaş",1)]
public string Adı { set; get; }
[Index("IX_Kişi_AdveYaş ",2)]
public int Yaş { set; get; }
```

§23.5.11 [Key] Özniteliği

Anahtar (**key**), bir veri tabanı tablosundaki her satırın (kaydın) benzersiz bir şekilde tanımlayan bir alandır. Varlık sınıfında **ID** içeren bir özellik yoksa bu nitelik kullanılabilir;

```
public class Personel
{
    [Key]
    public int SicilNo { set; get; }
    public string Adı { set; get; }
}
```

Aslında bu niteliği kullanmadan aşağıdaki gibi yapılabilirdi;

```
public class Personel
{
    public int PersonelId { set; get; } //Personel ID anahtar olarak görülür
    public string Adı { set; get; }
}
```

§23.5.12 [StringLength(int)] Özniteliği

Varlık nesnesinde metinlere karakter sınırlaması getirmek için kullanılır;

```
public class Personel
{
    public int PersonelId { set; get; }

    [StringLength(100)]
```

```
public string Adı { set; get; }
}
```

§23.5.13 [DatabaseGenerated] Özniteliği

Bazı durumlarda anahtar alanın otomatik artan bir kolon olması istenmez. Aşağıdaki `İl` varlığını düşünelim;

```
public class İl
{
    public int İlId { get; set; } // İlId veritabanında otomatik artıtılan (IDENTITY)
    public string İlAdı { get; set; }
}
```

Eğer il trafik kodlarını **anahtar** (key) olarak otomatik artmayan bir sütun istiyorsak varlığı aşağıdaki şekilde tanımlamalıyız;

```
public class İl
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int TrafikKodu { get; set; }
    public string İlAdı { get; set; }
}
```

Veri tabanında hesaplanan bir kolon varsa da bu nitelik kullanılabilir. Aşağıdaki gibi bir tablo oluşturmak istiyorsak;

```
CREATE TABLE [Personel] (
    [Adı] varchar(100) PRIMARY KEY,
    [DoğumTarihi] Date NOT NULL,
    [Yaş] AS DATEDIFF(year, [DoğumTarihi], GETDATE())
)
```

Bu tabloya ilişkin varlığı aşağıdaki gibi kodlamalıyız;

```
[Table("Personel")]
public class Kişi
{
    [Key, StringLength(100)]
    public string Adı { get; set; }
    public DateTime DoğumTarihi { get; set; }

    [DatabaseGenerated(DatabaseGeneratedOption.Computed)]
    public int Yaş { get; set; }
}
```

23.6 Mevcut Veri Tabanıyla Önce Kod

Önce kod (code first) yaklaşımıyla hali hazırda var olan tablolar içeren veri tabanı ile çalışılabilir. Bunun için içerisinde il listesi olan bir veri tabanını `SQLServerMevcut.mdf` örnek alalım. Veri tabanında aşağıdaki tablo var olsun;

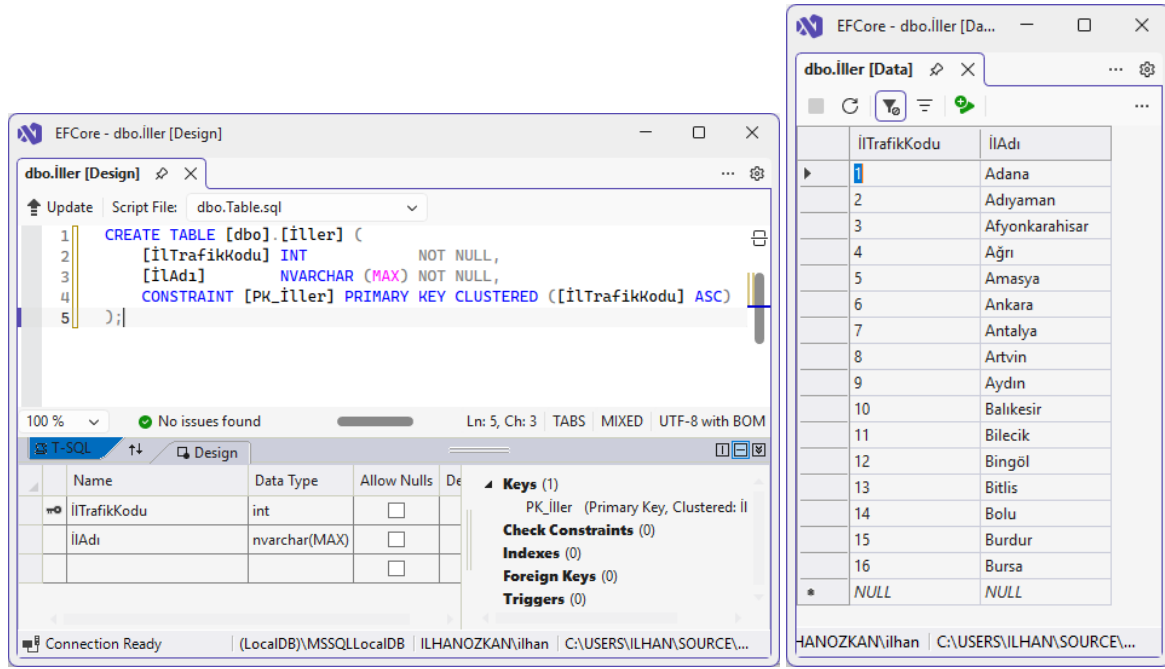
```
CREATE TABLE [dbo].[İller] (
    [İlTrafikKodu] INT NOT NULL,
    [İlAdı] NVARCHAR (MAX) NOT NULL,
    CONSTRAINT [PK_İller] PRIMARY KEY CLUSTERED ([İlTrafikKodu] ASC)
);
```

Bu tablonun içinde de il listemiz işlemiş olsun. Bu tabloyu kullanacak önce kod projesi için oluşacak uygulamanın yapısı aşağıda verilmiştir;

```
EFCoreUygulama/ → Çözüm (Solution)
├─ CodeFirst3/ → Proje (Console Application)
│   ├─ SQLServerMevcut.mdf → MSSQL Veritabanı (Data)
│   ├─ SQLServerMevcut_log.ldf → MSSQL Veritabanı (Log)
│   └─ Program.cs → Ana (Main) Program
```

Projeyi oluşturmak için;

1. **Visual Studio** üzerinde **File** → **New** → **Project...** seçilerek **EFCoreUygulama** adlı çözüm içinde **CodeFirst3**



Şekil 23.4: SQLServerMevcut.mdf Veritabanında Önceden Oluşturulan İller Tablosu

- adlı ilk projemizi **Console Application** olarak oluşturulur.
- Visual Studio** üzerinde **Tools**→**Connect To Database** tıklanarak *projenin klasöründe SQLServerMevcut.mdf* adında bir *Microsoft SQL Server LocalDB* veritabanı oluşturulur.
- Visual Studio** üzerinde **Tools** →**Server Explorer** açılarak 23.4 şeklinde gösterilen *tablo oluşturulur ve içerisine birkaç kayıt atılır. Maksat veritabanı önce hazırlanmış olsun.*
- Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır; *Microsoft.EntityFrameworkCore* ile *Microsoft.EntityFrameworkCore.SqlServer* paketleri yüklenir.
- Proje ile oluşan *Program.cs* dosyasını aşağıdaki şekilde değiştirilir;

```
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
var db = Path.Combine(projeKlasörü, "SQLServerMevcut.mdf");

string connectionString = $"Server=(LocalDB)\MSSQLLocalDB; AttachDbFilename={db};
Integrated Security=True;Connect Timeout=30;";
var options = new DbContextOptionsBuilder<AppDbContext>()
    .UseSqlServer(connectionString)
    .Options;

using (var context = new AppDbContext(options))
{
    var il = context.İller.FirstOrDefault();
    if (il != null)
    {
        Console.WriteLine($"İl: {il.İlAdı}, Trafik Kodu: {il.İlTrafikKodu}");
        //İl: Adana, Trafik Kodu: 1
    }
    else
    {
        Console.WriteLine("İl bulunamadı.");
    }
}
```

```

        il.İlAdı = "Adana 1 Numara"; //Veri Tabanında "Adana", "Adana 1 Numara" olarak
        ↪ değişecektir.
        context.SaveChanges();
        context.Database.CloseConnection();
    }

    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
        public DbSet<İl> İller { get; set; }
    }

    public class İl
    {
        [Key]
        public int İlTrafikKodu { get; set; }
        public string İlAdı { get; set; }
    }

```

6. Program çalıştırıldığında daha önce hazırlanmış ve içi dolu olan **İller** tablosundan veri çekerek gösterecektir. Ayrıca Veri Tabanında **İlTrafikKodu**, 1 olan "Adana", "Adana 1 Numara" olarak değişecektir.

23.7 Veri Göçü

Gerçek olarak uygulanan projelerde, özellikler uygulandıkça veri modelleri değişir: yeni varlıklar veya özellikler eklenir ve kaldırılır ve veri tabanı şemalarının uygulama ile senkronize tutulabilmesi için buna göre değiştirilmesi gerekir. EF Core'daki **göç (migration)** özelliği, veri tabanındaki verileri korurken veri tabanı şemasını uygulamanın veri modeliyle senkronize tutmak için artımlı olarak güncelleme bir yolunu sağlar.

EF Core'daki göç özelliğini kullanabilmek için; yukarıdaki gibi **daha önce EF Core** ile yönetilmiş **veri tabanı nesnelerinizi içeren** geçerli bir **DbContext** içeren bir uygulamanız olması gerekir. Göç özelliklerini kullanmak için EF Core ile yönetilen bir proje oluşturalım. Oluşacak uygulamanın yapısı aşağıda verilmiştir;

```

EFCoreUygulama/ → Çözüm (Solution)
├─ CodeFirst4/ → Proje (Console Application)
│   ├── DataBaseFirst.mdf → MSSQL Veritabanı (Data)
│   ├── DataBaseFirst_log.ldf → MSSQL Veritabanı (Log)
│   └─ Program.cs → Ana (Main) Program

```

Projeyi oluşturmak için;

1. Visual Studio üzerinde **File→New→Project...** seçilerek **EFCoreUygulama** adlı çözüm içinde **CodeFirst4** adlı ilk projemizi **Console Application** olarak oluşturulur.
2. **Solution Explorer** üzerinde projemize sağ tıklanarak **Add→New Item...** seçilir ve **DataBaseFirst.mdf** adında bir *Microsoft SQL Server LocalDB* veritabanı *proje klasöründe* oluşturulur.
3. **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage Nuget Packages...** tıklanır ve **Microsoft.EntityFrameworkCore** ve **Microsoft.EntityFrameworkCore.SqlServer** paketleri yüklenir.
4. Proje ile oluşan **Program.cs** dosyasını aşağıdaki şekilde değiştirilir;

```

using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

using (var context = new AppDbContext())
{
    context.Database.EnsureCreated();
    var iller = context.İller.ToList();
    Console.WriteLine("İller:");
    foreach (var il in iller)
    {
        Console.WriteLine($"Trafik Kodu: {il.İlTrafikKodu}, İl Adı: {il.İlAdı}");
    }
    context.Database.CloseConnection();
}

```

```

public class AppDbContext : DbContext
{
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
        var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
        var db = Path.Combine(projeKlasörü, "DataBaseFirst.mdf");
        string connectionString = $"Server=(LocalDB)\MSSQLLocalDB;
        ↪ AttachDbFilename={db};Integrated Security=True;Connect Timeout=30;";
        optionsBuilder.UseSqlServer(connectionString);
    }
    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Personel>()
            .HasOne(p => p.Adres)
            .WithMany()
            .HasForeignKey(p => p.PostaAdresID);
        modelBuilder.Entity<Adres>()
            .HasOne(a => a.İl)
            .WithMany()
            .HasForeignKey(a => a.İlTrafikKodu);
        modelBuilder.Entity<İl>().HasData(
            new İl { İlTrafikKodu = 1, İlAdı = "Adana" },
            new İl { İlTrafikKodu = 2, İlAdı = "Adıyaman" },
            new İl { İlTrafikKodu = 3, İlAdı = "Afyonkarahisar" },
            new İl { İlTrafikKodu = 4, İlAdı = "Ağrı" },
            new İl { İlTrafikKodu = 5, İlAdı = "Amasya" },
            new İl { İlTrafikKodu = 6, İlAdı = "Ankara" },
            new İl { İlTrafikKodu = 7, İlAdı = "Antalya" },
            new İl { İlTrafikKodu = 8, İlAdı = "Artvin" },
            new İl { İlTrafikKodu = 9, İlAdı = "Aydın" },
            new İl { İlTrafikKodu = 10, İlAdı = "Balıkesir" },
            new İl { İlTrafikKodu = 11, İlAdı = "Bilecik" },
            new İl { İlTrafikKodu = 12, İlAdı = "Bingöl" },
            new İl { İlTrafikKodu = 13, İlAdı = "Bitlis" },
            new İl { İlTrafikKodu = 14, İlAdı = "Bolu" },
            new İl { İlTrafikKodu = 15, İlAdı = "Burdur" },
            new İl { İlTrafikKodu = 16, İlAdı = "Bursa" }
        );
    }
}

public DbSet<Personel> Personel { get; set; }
public DbSet<İl> İller { get; set; }
}

public class Personel
{
    public int PersonelId { get; set; }
    [StringLength(100)]
    public string Adı { get; set; }
    public string Soyadı { get; set; }
    public int PostaAdresID { get; set; }
    [ForeignKey("PostaAdresID")]
    public virtual Adres Adres { get; set; }
}

public class Adres {
    public int AdresId { get; set; }
    [StringLength(200)] //Cadde-Sokak-Bulvar-Meydan
    public string CSBM { get; set; }
    [StringLength(10)]
    public string DışKapıNo { get; set; }
    [StringLength(10)]

```

```

    public string İçKapıNo { get; set; }
    [StringLength(100)]
    public string İlçe { get; set; }
    [StringLength(50)]
    public int İlTrafikKodu { get; set; }
    [ForeignKey("İlTrafikKodu")]
    public virtual İl İl { get; set; }
    public long PostaKodu { get; set; }
}
public class İl
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int İlTrafikKodu { get; set; }
    public string İlAdı { get; set; }
}

```

- Proje bir kez çalıştırılarak **EF Core** tarafından veri tabanında ilgili tablolar ve verilerin oluşması sağlanır. Burada dikkat edilmesi gereken **AppDbContext** sınıfının yapıcısını **parametresiz olarak kullanmaktır**. Bunu yapmak için **onConfiguring()** yöntemi bu nedenle bu sınıfa bulunur. Ayrıca il listesini de bir başka veritabanına göç sırasında oluşturmak istiyorsak **OnModelCreating()** yöntemini de ekleyip göç edilen veri tabanına istediğimiz verileri kaydedebiliriz.
- Projemizin ileride olabilecek yeni veritabanlarına göç etmesini istiyorsak; **Solution Explorer** üzerinde projemize sağ tıklanarak **Manage NuGet Packages...** tıklanır; **Microsoft.EntityFrameworkCore.Tools** ve **Microsoft.EntityFrameworkCore.Design** paketleri yüklenir.
- Ardından projemizde mevcut veri tabanının ve modelin ilk anlık görüntüsünü saklamak için; **Visual Studio** üzerinde **Tools**→**NuGet Package Manager**→**Package Manager Console** tıklanır ve açılan pencerede **Add-Migration Başlangıç** komutu yazılır ve çalıştırılır. Burada dikkat edilmesi gereken göç edilecek projenin varsayılan proje (**default project**) ayarının mevcut projemizi (**CodeFirst4**) göstermesidir;


```

PM> Add-Migration Başlangıç
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
.NET Core CLI ile oluşturulan projelerde bu işlemi yapabilmek için proje klasöründe aşağıdaki komut çalıştırılır.
$ dotnet ef migrations add Başlangıç

```
- Komut sonrasında projemizde **Migrations** adında klasör oluşur. Bu klasör içinde veritabanının anlık görüntüsünden yeni bir veritabanı oluşturacak olan **20250530083726_Başlangıç.cs** (Dosya Adı:Zaman damgası ve Göç adı) adlı göç dosyası oluşur. Ayrıca varlıklarımızın o anki bir fotoğrafı olan **AppDbContextModelSnapshot.cs** dosyası bulunur. Projenin yapısı aşağıdaki gibi olur;

```

EFSQLUygulama/ → Çözüm (Solution)
├── CodeFirst4/ → Proje (Console Application))
│   ├── Migrations/ → Klasör
│   │   ├── 20250530122325_Başlangıç.cs → Zaman Damgalı Göç
│   │   ├── AppDbContextModelSnapshot.cs → AppDbContextModel Yedeği
│   │   ├── DataBase4.mdf → MSSQL Veritabanı (Data)
│   │   ├── DataBase4.ldf → MSSQL Veritabanı (Log)
│   │   └── Program.cs → Ana (Main) Program

```

- Migrations** klasöründe oluşan ilk dosya **20260429054001_Başlangıç.cs** göç dosyası aslında anlık görüntü alınan veritabanını, bir başka veri tabanına geçtiğimizde bu anlık görüntüyü yeni veritabanında oluşturacak kod dosyasıdır. Şimdi **20260429054001_Başlangıç.cs** göç dosyasının içeriğini inceleyelim;

```
using Microsoft.EntityFrameworkCore.Migrations;
```

```
#nullable disable
```

```
#pragma warning disable CA1814 // Prefer jagged arrays over multidimensional
```



```

namespace CodeFirst4.Migrations
{
    /// <inheritdoc />
    public partial class Başlangıç : Migration
    {
        /// <inheritdoc />
        protected override void Up(MigrationBuilder migrationBuilder)
        {
            migrationBuilder.CreateTable(
                name: "İller",
                columns: table => new
                {
                    İlTrafikKodu = table.Column<int>(type: "int", nullable: false),
                    İlAdı = table.Column<string>(type: "nvarchar(max)", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_İller", x => x.İlTrafikKodu);
                });

            migrationBuilder.CreateTable(
                name: "Adres",
                columns: table => new
                {
                    AdresId = table.Column<int>(type: "int", nullable: false)
                        .Annotation("SqlServer:Identity", "1, 1"),
                    CSBM = table.Column<string>(type: "nvarchar(200)", maxLength: 200,
                        nullable: false),
                    DışKapıNo = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
                        nullable: false),
                    İçKapıNo = table.Column<string>(type: "nvarchar(10)", maxLength: 10,
                        nullable: false),
                    İlçe = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
                        nullable: false),
                    İlTrafikKodu = table.Column<int>(type: "int", maxLength: 50, nullable:
                        false),
                    PostaKodu = table.Column<long>(type: "bigint", nullable: false)
                },
                constraints: table =>
                {
                    table.PrimaryKey("PK_Adres", x => x.AdresId);
                    table.ForeignKey(
                        name: "FK_Adres_İller_İlTrafikKodu",
                        column: x => x.İlTrafikKodu,
                        principalTable: "İller",
                        principalColumn: "İlTrafikKodu",
                        onDelete: ReferentialAction.Cascade);
                });

            migrationBuilder.CreateTable(
                name: "Personel",
                columns: table => new
                {
                    PersonelId = table.Column<int>(type: "int", nullable: false)
                        .Annotation("SqlServer:Identity", "1, 1"),
                    Adı = table.Column<string>(type: "nvarchar(100)", maxLength: 100,
                        nullable: false),
                    Soyadı = table.Column<string>(type: "nvarchar(max)", nullable: false),
                    PostaAdresID = table.Column<int>(type: "int", nullable: false)
                },
                constraints: table =>
                {

```

```

        table.PrimaryKey("PK_Personel", x => x.PersonelId);
        table.ForeignKey(
            name: "FK_Personel_Adres_PostaAdresID",
            column: x => x.PostaAdresID,
            principalTable: "Adres",
            principalColumn: "AdresId",
            onDelete: ReferentialAction.Cascade);
    });

    migrationBuilder.InsertData(
        table: "İller",
        columns: new[] { "İlTrafikKodu", "İlAdı" },
        values: new object[,]
        {
            { 1, "Adana" },
            { 2, "Adıyaman" },
            { 3, "Afyonkarahisar" },
            { 4, "Ağrı" },
            { 5, "Amasya" },
            { 6, "Ankara" },
            { 7, "Antalya" },
            { 8, "Artvin" },
            { 9, "Aydın" },
            { 10, "Balıkesir" },
            { 11, "Bilecik" },
            { 12, "Bingöl" },
            { 13, "Bitlis" },
            { 14, "Bolu" },
            { 15, "Burdur" },
            { 16, "Bursa" }
        });

    migrationBuilder.CreateIndex(
        name: "IX_Adres_İlTrafikKodu",
        table: "Adres",
        column: "İlTrafikKodu");

    migrationBuilder.CreateIndex(
        name: "IX_Personel_PostaAdresID",
        table: "Personel",
        column: "PostaAdresID");
}

/// <inheritdoc />
protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropTable(
        name: "Personel");

    migrationBuilder.DropTable(
        name: "Adres");

    migrationBuilder.DropTable(
        name: "İller");
}
}

```

Gördüğünüz gibi, `Up()` yöntemi `İller`, `Adres` ve `Personel` adlı üç tablo ve alanları oluşturulur. Ek olarak, tablolar arasındaki ilişkiler ve veritabanı dizinler (`index`) de oluşturulur. Diğer tarafta `Down()` yöntemi göç aktivitelerini (`Up` yöntemi ile yapılanları) tersine çevirmeye çalışır. Göçünüz konusunda kendinize güveniyorsanız, göçü istediğiniz veri tabanına uygulayabilirsiniz.

10. `Migrations` klasöründe oluşan ikinci dosya `AppDbContextModelSnapshot.cs` dosyası da mevcut varlık-

larımızın anlık görüntüsünü tutan bir dosyadır. Bir başka veri tabanına geçtiğimizde veritabanını bu anlık görüntüsünü ile kullanmak için oluşturulur. Şimdi `AppDbContextModelSnapshot.cs` göç dosyasının içeriğini inceleyelim;

```
// <auto-generated />
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Infrastructure;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;

#nullable disable

namespace CodeFirst4.Migrations
{
    [DbContext(typeof(AppDbContext))]
    partial class AppDbContextModelSnapshot : ModelSnapshot
    {
        protected override void BuildModel(ModelBuilder modelBuilder)
        {
            #pragma warning disable 612, 618
            modelBuilder
                .HasAnnotation("ProductVersion", "10.0.7")
                .HasAnnotation("Relational:MaxIdentifierLength", 128);

            SqlServerModelBuilderExtensions.UseIdentityColumns(modelBuilder);

            modelBuilder.Entity("Adres", b =>
            {
                b.Property<int>("AdresId")
                    .ValueGeneratedOnAdd()
                    .HasColumnType("int");

                SqlServerPropertyBuilderExtensions.UseIdentityColumn(b.Property<int>("AdresId"));

                b.Property<string>("CSBM")
                    .IsRequired()
                    .HasMaxLength(200)
                    .HasColumnType("nvarchar(200)");

                b.Property<string>("DışKapıNo")
                    .IsRequired()
                    .HasMaxLength(10)
                    .HasColumnType("nvarchar(10)");

                b.Property<long>("PostaKodu")
                    .HasColumnType("bigint");

                b.Property<int>("İlTrafikKodu")
                    .HasMaxLength(50)
                    .HasColumnType("int");

                b.Property<string>("İlçe")
                    .IsRequired()
                    .HasMaxLength(100)
                    .HasColumnType("nvarchar(100)");

                b.Property<string>("İçKapıNo")
                    .IsRequired()
                    .HasMaxLength(10)
                    .HasColumnType("nvarchar(10)");

                b.HasKey("AdresId");
            });
        }
    }
}
```

```
b.HasIndex("İlTrafikKodu");

b.ToTable("Adres");
});

modelBuilder.Entity("Personel", b =>
{
    b.Property<int>("PersonelId")
        .ValueGeneratedOnAdd()
        .HasColumnType("int");

    SqlServerPropertyBuilderExtensions
        .UseIdentityColumn(b.Property<int>("PersonelId"));

    b.Property<string>("Adı")
        .IsRequired()
        .HasMaxLength(100)
        .HasColumnType("nvarchar(100)");

    b.Property<int>("PostaAdresID")
        .HasColumnType("int");

    b.Property<string>("Soyadı")
        .IsRequired()
        .HasColumnType("nvarchar(max)");

    b.HasKey("PersonelId");

    b.HasIndex("PostaAdresID");

    b.ToTable("Personel");
});

modelBuilder.Entity("İl", b =>
{
    b.Property<int>("İlTrafikKodu")
        .HasColumnType("int");

    b.Property<string>("İlAdı")
        .IsRequired()
        .HasColumnType("nvarchar(max)");

    b.HasKey("İlTrafikKodu");

    b.ToTable("İller");

    b.HasData(
        new
        {
            İlTrafikKodu = 1,
            İlAdı = "Adana"
        },
        new
        {
            İlTrafikKodu = 2,
            İlAdı = "Adıyaman"
        },
        new
        {
            İlTrafikKodu = 3,
            İlAdı = "Afyonkarahisar"
        },
    );
});
```

```
new
{
    İlTrafikKodu = 4,
    İlAdı = "Ağrı"
},
new
{
    İlTrafikKodu = 5,
    İlAdı = "Amasya"
},
new
{
    İlTrafikKodu = 6,
    İlAdı = "Ankara"
},
new
{
    İlTrafikKodu = 7,
    İlAdı = "Antalya"
},
new
{
    İlTrafikKodu = 8,
    İlAdı = "Artvin"
},
new
{
    İlTrafikKodu = 9,
    İlAdı = "Aydın"
},
new
{
    İlTrafikKodu = 10,
    İlAdı = "Balıkesir"
},
new
{
    İlTrafikKodu = 11,
    İlAdı = "Bilecik"
},
new
{
    İlTrafikKodu = 12,
    İlAdı = "Bingöl"
},
new
{
    İlTrafikKodu = 13,
    İlAdı = "Bitlis"
},
new
{
    İlTrafikKodu = 14,
    İlAdı = "BoLu"
},
new
{
    İlTrafikKodu = 15,
    İlAdı = "Burdur"
},
new
{

```

```

        İlTrafikKodu = 16,
        İlAdı = "Bursa"
    });

modelBuilder.Entity("Adres", b =>
{
    b.HasOne("İl", "İl")
        .WithMany()
        .HasForeignKey("İlTrafikKodu")
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();

    b.Navigation("İl");
});

modelBuilder.Entity("Personel", b =>
{
    b.HasOne("Adres", "Adres")
        .WithMany()
        .HasForeignKey("PostaAdresID")
        .OnDelete(DeleteBehavior.Cascade)
        .IsRequired();

    b.Navigation("Adres");
});
#pragma warning restore 612, 618
}
}
}
}

```

11. Şimdi mevcut projemizde veritabanını değiştirerek mevcut projemizde kullandığımız veri göçü yapacağız. Mevcut projemizi açıp;
12. **Visual Studio** üzerinde projemize sağ tıklanarak **Tools→Connect To Database...** seçilir ve göç edilecek **DatabaseGoc.mdf** adında bir *Microsoft SQL Server LocalDB* veritabanı *proje klasöründe* oluşturulur.
13. **Program.cs** içinde **yeni veri tabanı bağlantımızı sağlayacak** değişikliği **AppDbContext** sınıfında veri tabanına bağlanmayı sağlayan **onConfiguring()** yönteminde yapıyoruz.

```

protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
    var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
    var db = Path.Combine(projeKlasörü, "DatabaseGoc.mdf"); // Göç edilecek Veritabanı
    string connectionString = $"Server=(LocalDB)\MSSQLLocalDB; AttachDbFilename={db};
    ↳ Integrated Security=True;Connect Timeout=30;";
    optionsBuilder.UseSqlServer(connectionString);
}

```
14. **Visual Studio** üzerinde **Tools→NuGet Package Manager→Package Manager Console** tıklanır ve açılan pencerede **Update-Database** komutu yazılır ve çalıştırılır;

```

PM> Update-Database
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See ...
Applying migration '20260429054001_Başlangıç'.
Done.
PM>

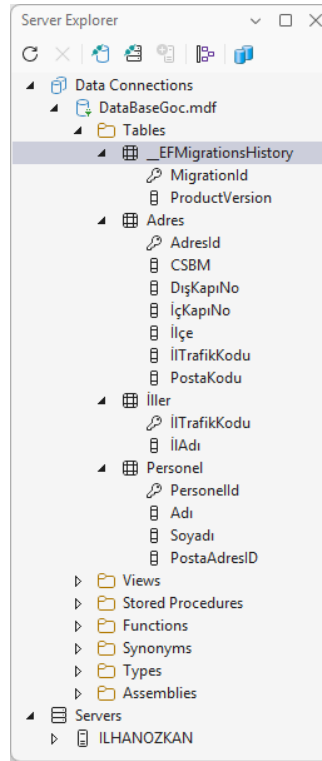
```
15. Artık projemiz için yeni veri tabanında **EF Core** için tablolar otomatik olarak oluşturmuştur (Şekil 23.5). **.NET Core CLI** ile oluşturulan projelerde bu işlemi yapabilmek için **proje klasöründe** aşağıdaki komut çalıştırılır.

```

$ dotnet ef database update

```

Bir **EF Core** projesinde **göç (migration)** işlemleri şu şekilde kullanılır;



Şekil 23.5: Göç Sonrası Veri Tabanı

- ◊ Bir veri modeli değişikliği yapıldığında, geliştirici veri tabanı şemasını senkronize tutmak için gerekli güncellemeleri açıklayan karşılık gelen bir göç eklemek için **EF Core Tools** kullanır. **EF Core**, farklılıkları belirlemek için geçerli modeli eski modelin bir anlık görüntüsüyle karşılaştırır ve göç kaynak dosyaları oluşturur; dosyalar projenizin kaynak denetiminde diğer kaynak dosyaları gibi izlenebilir.
- ◊ Yeni bir göç oluşturulduktan sonra, çeşitli şekillerde bir veri tabanına uygulanabilir. EF Core, uygulanan tüm göçleri özel bir geçmiş tablosunda kaydeder ve hangi göçlerin uygulandığını ve hangilerinin uygulanmadığını bilmesini sağlar.
- ◊ **Add-Migration** ve **Update-Database** komutları için aktiviteleri ayarlamak için kullanılabilecek çeşitli seçenekler mevcuttur. Bunları görmek ve incelemek için;
`get-help Add-Migration` ve
`get-help Update-Database` komutları kullanılabilir.

Veriyle Besleme

Geliştirmenin yapıldığı ortamın dışında çalışan uygulamalar genellikle veri tabanı güncellemeleri (**update**) gerektirir. Veri tabanı yamalarını (**patch**) oluşturmak için **AddMigration** komutunu kullandıktan sonra, güncellemeleri test ve diğer ortamlarda çalıştırmanız gerekir. Bunun yanında uygulamanın müşteri ortamına bağlantı izni yok ya da bu ortamda **Visual Studio** kurulmamış ise bu komutu çalıştıramayız. Bunun çözüm örneğimizde olduğu gibi **DbContext** sınıfına **OnModelCreating** yöntemini eklemektir. Göç sırasında yeni veri tabanında tabloları verilerle doldurma işlemine **veriyle besleme** (**seed-ing data**) adı verilir.

Şu ana kadar anlatılan göç işlemleri **Package Manager Console** açılarak **Add-Migration Başlangıç** ve **Update-Database** gibi komutları elle girerek yaptık. Bunu bir yardımcı program yazarak otomatik olarak da yapabiliriz. `context.Database.Migrate()` yöntemi, **EF Core** içerisinde, önce kod (kızılcode first) yaklaşımıyla oluşturulan model değişikliklerini yani göç (migration) dosyalarını veritabanına otomatik olarak uygulamak için kullanılır. Aşağıda örneği verilmiştir;

```
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

// Main Metodu İçeriği
try
{

```



```

using (var context = new AppDbContext())
{
    Console.WriteLine("Veritabanı kontrol ediliyor/güncelleniyor...");

    // EnsureCreated() yerine Migrate() kullanıyoruz. Bu komut bekleyen migration varsa
    // → uygular, veritabanı yoksa oluşturur.
    context.Database.Migrate();

    var iller = context.İller.ToList();
    Console.WriteLine("\nKayıtlı İller:");
    foreach (var il in iller)
    {
        Console.WriteLine($"Trafik Kodu: {il.İlTrafikKodu}, İl Adı: {il.İlAdı}");
    }
}
}
catch (Exception ex)
{
    Console.WriteLine($"Bir hata oluştu: {ex.Message}");
}

public class AppDbContext : DbContext
{
    public DbSet<Personel> Personel { set; get; }
    public DbSet<İl> İller { set; get; }
    public DbSet<Adres> Adresler { set; get; } // Adres tablosu eksikti, eklendi.

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        var çalışmaKlasörü = AppDomain.CurrentDomain.BaseDirectory;
        var projeKlasörü = Path.GetFullPath(Path.Combine(çalışmaKlasörü, @"..\..\..\"));
        var db = Path.Combine(projeKlasörü, "DataBaseGoc.mdf");
        string connectionString = $"Server=(LocalDB)\\MSSQLLocalDB;
        → AttachDbFilename={db};Integrated Security=True;Connect
        → Timeout=30;TrustServerCertificate=True;";
        optionsBuilder.UseSqlServer(connectionString);
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Fluent API ilişkileri
        modelBuilder.Entity<Personel>()
            .HasOne(p => p.Adres)
            .WithMany()
            .HasForeignKey(p => p.PostaAdresID);

        modelBuilder.Entity<Adres>()
            .HasOne(a => a.İl)
            .WithMany()
            .HasForeignKey(a => a.İlTrafikKodu);

        // Seed Data (Örnek Veriler)
        modelBuilder.Entity<İl>().HasData(
            new İl { İlTrafikKodu = 1, İlAdı = "Adana" },
            new İl { İlTrafikKodu = 6, İlAdı = "Ankara" },
            new İl { İlTrafikKodu = 34, İlAdı = "İstanbul" }
        );
    }
}

public class Personel
{

```

```

public int PersonelId { get; set; }
[StringLength(100)]
public string Adı { get; set; }
public string Soyadı { get; set; }
public int PostaAdresID { get; set; }
[ForeignKey("PostaAdresID")]
public virtual Adres Adres { get; set; }
}

public class Adres {
    public int AdresId { get; set; }
    [StringLength(200)]
    public string CSBM { get; set; }
    [StringLength(10)]
    public string DışKapıNo { get; set; }
    [StringLength(10)]
    public string İçKapıNo { get; set; }
    [StringLength(100)]
    public string İlçe { get; set; }

    public int İlTrafikKodu { get; set; }

    [ForeignKey("İlTrafikKodu")]
    public virtual İl İl { get; set; }
    public long PostaKodu { get; set; }
}

public class İl
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.None)]
    public int İlTrafikKodu { get; set; }
    public string İlAdı { get; set; }
}

```

Bu yöntem, özellikle üretim (**production**) ortamlarında veya CI/CD süreçlerinde, **Package Manager Console** üzerinden **Update-Database** gibi manuel komut çalıştırmak yerine, uygulamanın açılışında veritabanı şemasını güncel tutmak için kullanılır. Veritabanında **__EFMigrationsHistory** tablosunu kontrol eder, projedeki göç dosyalarıyla karşılaştırır ve eksik olanları çalıştırarak veritabanı şemasını (tablolar, sütunlar vb.) günceller. Manuel SQL yazma veya CLI komutları çalıştırma ihtiyacını ortadan kaldırır. **MigrateAsync()** yöntemi asenkron çalışan versiyonudur. Performansı artırmak için tercih edilebilir. Çoklu sunucu ortamlarında (**load balancer**) aynı anda birden fazla örnek çalışıyorsa, göç (**migration**) çakışmalarını önlemek için dikkatli kullanılmalıdır.

23.8 Düşün ve Çöz

- ◊ **Düşün:** ORM (**Object-Relational Mapping**) nedir ve veri erişim katmanını neden kolaylaştırır? ADO.NET ile doğrudan SQL yazmaya kıyasla Entity Framework'ün getirdiği soyutlama maliyeti var mıdır?
- ◊ **Düşün:** 'Önce kod' (**code first**) ile 'önce veri tabanı' (**database first**) yaklaşımları arasındaki farklar nelerdir? Hangi senaryoda hangisi tercih edilir? Mevcut bir veri tabanının olduğu ve mevcut bir kod tabanının olduğu iki farklı senaryo düşününüz.
- ◊ **Düşün:** **DbContext** sınıfının yaşam döngüsü (**lifetime**) neden önemlidir? Bir Web API'de **DbContext**'i **Singleton** olarak kaydetmenin neden sorun yarattığını; **Scoped** olarak kaydetmenin neden doğru olduğunu açıklayınız.
- ◊ **Düşün:** EF Core'un değişiklik takibi (**change tracking**) mekanizması nasıl çalışır? **AsNoTracking()** kullanmanın salt okunur sorgularda performans avantajı nedir? Her zaman **AsNoTracking()** kullanmak doğru mudur?
- ◊ **Düşün:** Göç (**migrations**) sistemi veri tabanı şemasını kodla yönetmenizi sağlar. Bir ekip projesinde göç çakışmaları nasıl oluşur ve nasıl çözülür? Göçü geri alma (**rollback**) işlemi nasıl yapılır?
- ◊ **Düşün:** EF Core'da **Eager Loading** (Include), **Lazy Loading** ve **Explicit Loading** arasındaki farklar nelerdir? N+1 sorgusunu önlemek için hangisi tercih edilir? **Lazy Loading**'in avantajı ve tuzağı nedir?

- ◇ **Düşün:** EF Core ile ham SQL sorgusu (`FromSqlRaw`, `ExecuteSqlRaw`) ne zaman kullanılır? ORM'nin ürettiği SQL'den daha performanslı özel sorgular yazmayı ne zaman düşünmelisiniz?
- ◇ **Çöz:** Öğrenci, Ders ve Kayıt tablolarını içeren bir okul veri tabanı oluşturun (`code first`, SQLite). Göç uygulayın. Bir öğrencinin tüm derslerini listeleyen, yeni ders kaydı yapan ve kaydı silen CRUD yöntemleri yazınız.
- ◇ **Çöz:** EF Core ile bir ürün kataloğu uygulaması geliştirin: Ürün ve Kategori arasında bire-çok ilişki kurun. **Eager Loading** (Include) kullanarak bir kategorideki tüm ürünleri getirin; **AsNoTracking** ile performansı karşılaştırın.
- ◇ **Çöz:** EF Core değişiklik takibini (`change tracking`) gösteren bir senaryo oluşturun: bir nesneyi okuyun, özelliğini değiştirin, SaveChanges öncesinde ChangeTracker.Entries() ile değişiklikleri listeleyin, ardından kaydedin.
- ◇ **Çöz:** Bir veritabanı tohum verisi (`seed data`) sistemi tasarlayın: `DbContext.OnModelCreating()` içinde Ürün ve Kategori tablolarını örnek verilerle doldurun; her migration uygulandığında bu veriler yoksa eklensin.
- ◇ **Çöz:** EF Core ile bir sipariş yönetimi uygulaması yazın: `Müşteri`, `Sipariş` ve `SiparişKalemi` arasında ilişkiler kurun. İş (`transaction`) kullanarak bir siparişi tüm kalemleriyle atomik olarak ekleyin; herhangi bir kalemde hata olursa geri alma (`rollback`) yapın.